



Traitement et Analyse de Données

PREPROCESSING

Cours 1 : Fondamentaux et Exploration des Données

Tacko CISS

Étudiante en Master 1 Systèmes d'Information

Table des matières

1	Introduction au Preprocessing	3
1.1	Objectifs du cours	3
1.1.1	Objectifs spécifiques	3
1.2	Pourquoi le preprocessing est crucial ?	3
1.2.1	Impact sur les performances	3
1.2.2	Statistiques clés	4
1.3	Le preprocessing dans le pipeline ML	4
1.4	Les 5 piliers du preprocessing	5
2	Comprendre les Données	6
2.1	Information, Donnée et Indicateur	6
2.1.1	Définitions fondamentales	6
2.1.2	Chaîne de transformation	7
2.2	Types de variables	7
2.2.1	Vue d'ensemble	7
2.2.2	Variables qualitatives (catégorielles)	7
2.2.3	Variables quantitatives (numériques)	8
2.2.4	Autres types de variables	9
2.3	Caractéristiques d'une variable	9
2.3.1	Les 3 dimensions	9
2.3.2	Rôles des variables dans l'analyse	9
2.4	Structures de données	9
2.4.1	Du scalaire au tenseur	10
2.4.2	DataFrame : la structure centrale	10
2.4.3	Représentation mathématique d'une matrice	10
3	Traitement des Données	11
3.1	Actions de preprocessing	11
3.1.1	Objectifs du traitement	11
3.2	Description et visualisation des données	11
3.2.1	Pourquoi explorer avant de nettoyer ?	12
3.2.2	Description statistique de base	12
3.2.3	Code Python : Description statistique	12
3.3	Visualisation des données	13
3.3.1	Visualisation des valeurs manquantes	13
3.3.2	Visualisation des distributions	14
3.3.3	Visualisation des valeurs aberrantes (outliers)	15
3.4	Les Données Aberrantes (Outliers) - Guide Complet	16

3.4.1	Types de données aberrantes	16
3.4.2	Exemples concrets de données aberrantes	16
3.4.3	Méthodes de détection des outliers	17
3.4.4	Traitement des outliers	19
3.5	L'Encodage des Variables Catégorielles - Guide Complet	20
3.5.1	Types d'encodage	20
3.5.2	Label Encoding - Encodage par étiquettes	20
3.5.3	One-Hot Encoding - Encodage binaire	22
3.5.4	Comparaison et choix de la méthode	23
3.5.5	Autres méthodes d'encodage avancées	23
3.5.6	Exemple complet : Pipeline d'encodage	24
3.5.7	Visualisation récapitulative de l'encodage	25
3.5.8	Visualisation des corrélations	25
3.5.9	Visualisation des incohérences	26
3.6	Checklist d'exploration	27
3.7	Outils et bibliothèques recommandées	27
4	Résumé et Points Clés	28
4.1	Ce qu'il faut retenir	28
4.2	Points clés à retenir	28

Chapitre 1

Introduction au Preprocessing

1.1 Objectifs du cours

🎯 Objectif

Objectif principal : Maîtriser l'ensemble des techniques de prétraitement des données pour préparer efficacement les données brutes en vue de l'apprentissage automatique.

1.1.1 Objectifs spécifiques

À l'issue de ce cours, vous serez capable de :

- ✔ **Comprendre** l'importance du preprocessing dans le pipeline de Machine Learning
- ✔ **Identifier** et traiter les problèmes de qualité des données
- ✔ **Appliquer** les techniques de nettoyage, transformation et encodage
- ✔ **Choisir** les méthodes appropriées selon le type de données et l'objectif

1.2 Pourquoi le preprocessing est crucial ?

1.2.1 Impact sur les performances

⚠ Important

Garbage in, garbage out — Des données de mauvaise qualité produisent invariablement de mauvais modèles, quelle que soit la sophistication de l'algorithme utilisé.

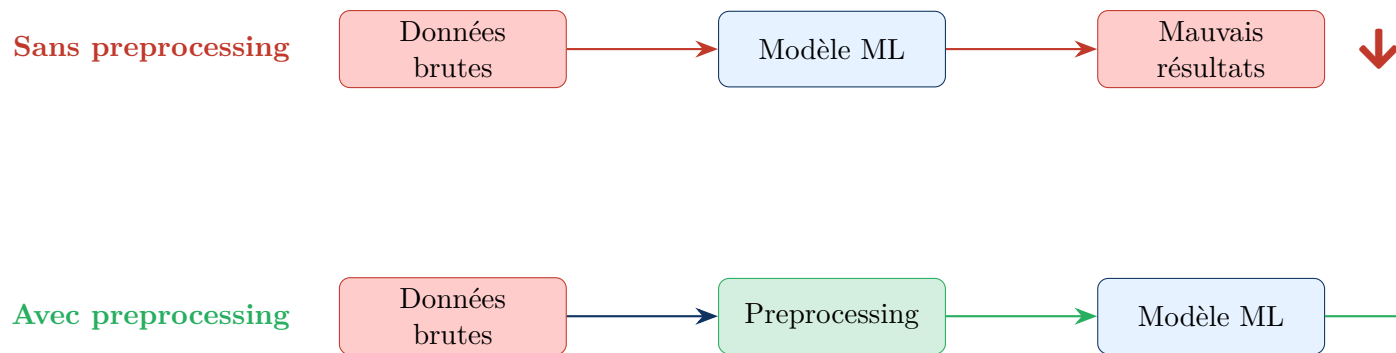
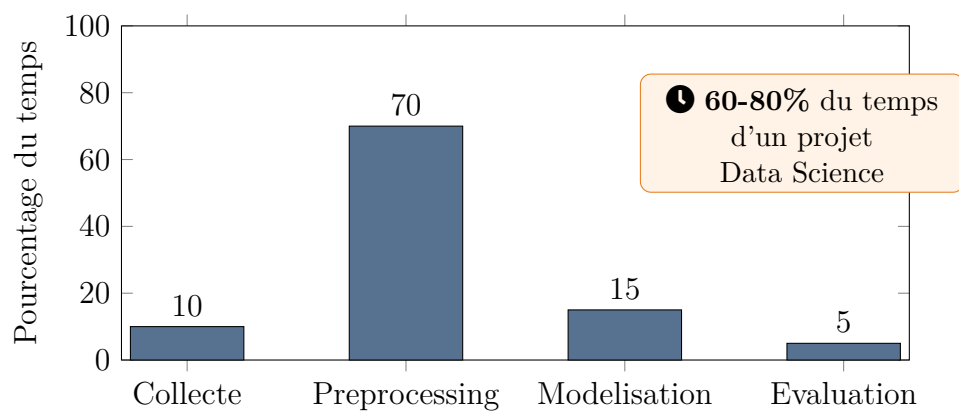


FIGURE 1.1 – Impact du preprocessing sur la qualité des résultats

1.2.2 Statistiques clés



1.3 Le preprocessing dans le pipeline ML

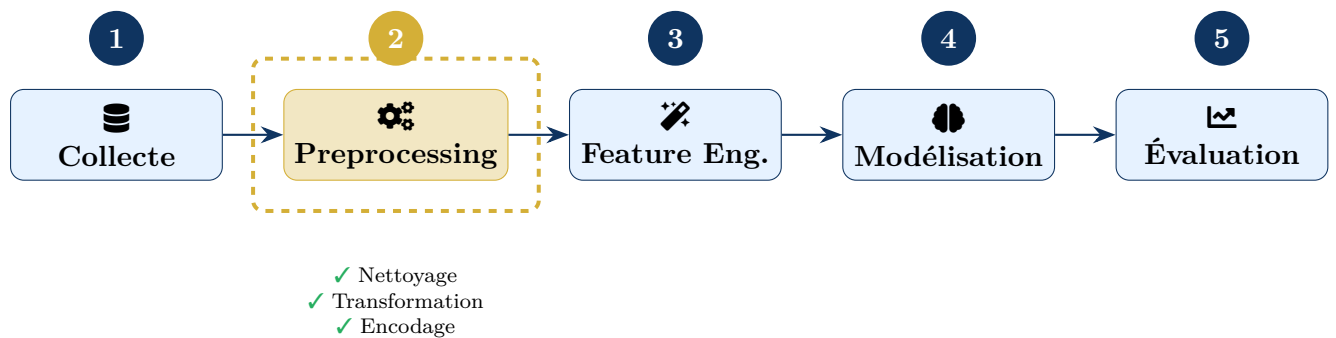


FIGURE 1.2 – Pipeline complet du Machine Learning

1.4 Les 5 piliers du preprocessing

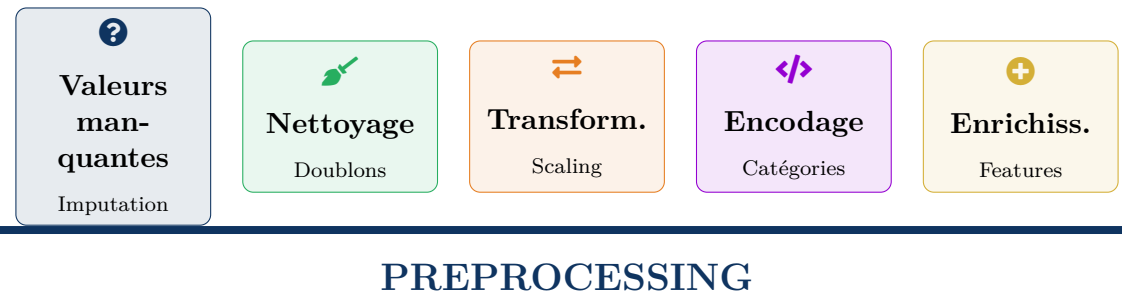


FIGURE 1.3 – Les 5 piliers fondamentaux du preprocessing

Chapitre 2

Comprendre les Données

2.1 Information, Donnée et Indicateur

2.1.1 Définitions fondamentales

Information

Information : Connaissance ou fait qui a du sens dans un contexte donné.

- **Caractéristique** : Interprétable par l'humain, contextualisée
- **Exemple** : "La température est élevée aujourd'hui"

Donnée

Donnée : Représentation brute, non interprétée, d'une information.

- **Caractéristique** : Format structuré, exploitable par machine
- **Exemple** : 28.5 (degré Celsius)

Indicateur

Indicateur : Mesure calculée à partir de données pour évaluer une situation.

- **Caractéristique** : Agrégation, comparaison, suivi dans le temps
- **Exemple** : Température moyenne mensuelle : 25°C

2.1.2 Chaîne de transformation

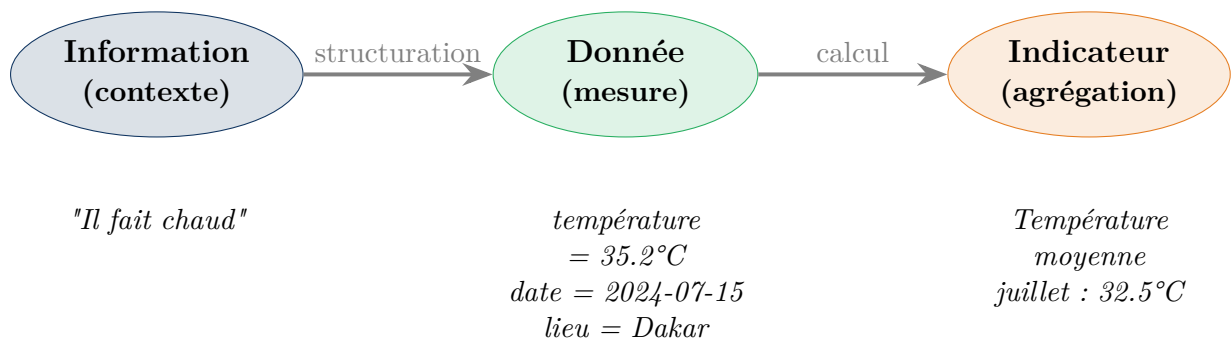


FIGURE 2.1 – De l'information à l'indicateur

Astuce

En Machine Learning, on travaille avec des **données** qu'on transforme en **indicateurs** (features) pour créer des modèles qui produisent de nouvelles **informations** (prédictions).

2.2 Types de variables

2.2.1 Vue d'ensemble

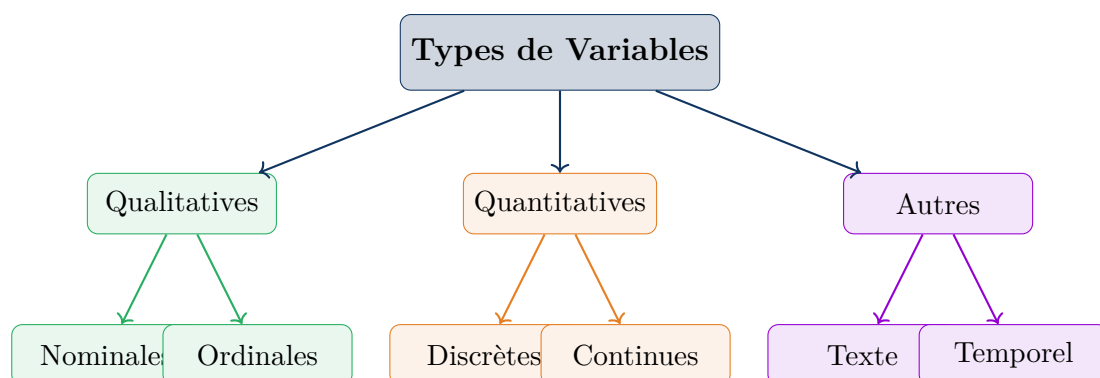


FIGURE 2.2 – Classification des types de variables

2.2.2 Variables qualitatives (catégorielles)

Définition

Variables qui décrivent des **catégories**, des **classes** ou des **attributs** non mesurables numériquement.

Type	Description	Exemples
Nominales	Catégories sans ordre naturel	Couleur (rouge, bleu), Ville (Paris, Dakar)
Ordinales	Catégories avec ordre naturel	Niveau (Débutant, Intermédiaire, Avancé)

TABLE 2.1 – Types de variables qualitatives

Exemple

- **Nominale** : Genre musical préféré (Rock, Jazz, Pop) — pas d'ordre logique
- **Ordinale** : Taille de vêtement (S < M < L < XL) — ordre naturel mais pas d'écart mesurable

2.2.3 Variables quantitatives (numériques)

Définition

Variables qui **mesurent des quantités** et permettent des opérations mathématiques.

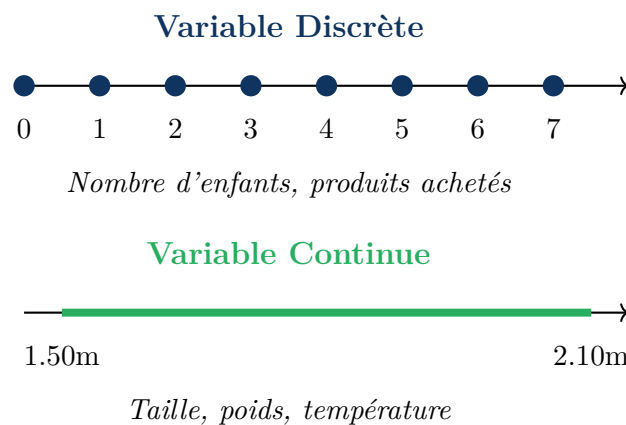


FIGURE 2.3 – Différence entre variables discrètes et continues

2.2.4 Autres types de variables

Icône	Type	Exemples	Traitement
	Textuelles	Commentaires, descriptions	NLP, tokenisation
	Temporelles	Dates, timestamps	Extraction de features
	Images	Photos, scans	CNN, tenseurs
	Audio	Sons, voix	Spectrogrammes
	Géospatiales	GPS, polygones	Coordonnées

TABLE 2.2 – Variables complexes et leurs traitements

2.3 Caractéristiques d'une variable

2.3.1 Les 3 dimensions

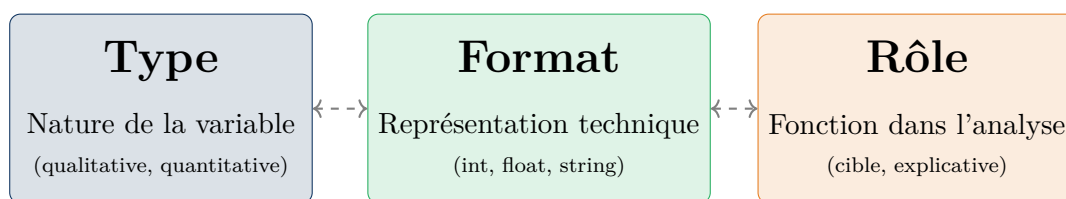


FIGURE 2.4 – Les trois dimensions d'une variable

💡 Exemple complet

Pour la variable **age** :

- **Type** : Quantitative continue
- **Format** : `float64` (Python) ou `REAL` (SQL)
- **Rôle** : Variable explicative (feature)
- **Domaine** : $[0, 120]$ années

2.3.2 Rôles des variables dans l'analyse

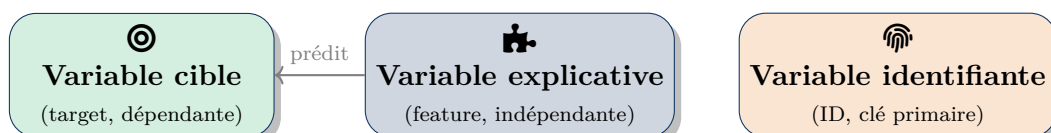


FIGURE 2.5 – Différents rôles des variables

2.4 Structures de données

2.4.1 Du scalaire au tenseur

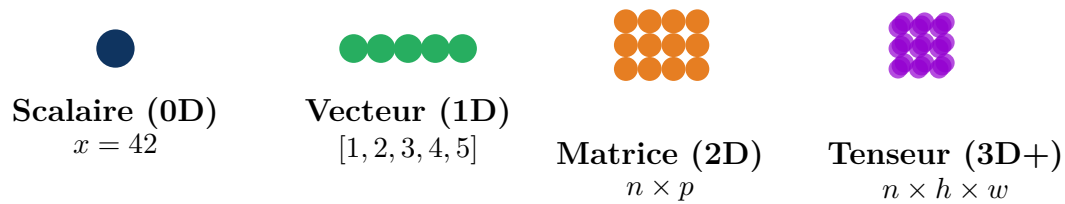


FIGURE 2.6 – Évolution des structures de données

2.4.2 DataFrame : la structure centrale

The diagram shows a DataFrame table with the following structure:

	Index	Nom	Âge	Ville	Salaire
Lignes	0	Alice	28	Paris	45000
	1	Bob	35	Lyon	52000
	2	Clara	42	Dakar	38000

Below the table, data types are indicated: Index (int), Nom (str), Âge (int), Ville (str), Salaire (float). An orange double-headed arrow above the table spans the columns from 'Nom' to 'Salaire', labeled 'Colonnes (variables)'.

FIGURE 2.7 – Structure d'un DataFrame

2.4.3 Représentation mathématique d'une matrice

Une matrice de données $\mathbf{X}$ avec n observations et p variables s'écrit :

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix}_{n \times p} \quad (2.1)$$

Où :

- n = nombre d'observations (lignes)
- p = nombre de variables (colonnes)
- x_{ij} = valeur de la variable j pour l'observation i

Chapitre 3

Traitement des Données

3.1 Actions de preprocessing

Important

Le preprocessing représente jusqu'à **80% du travail** dans un projet de Data Science. Une préparation rigoureuse des données est essentielle pour la réussite du projet.

3.1.1 Objectifs du traitement

Il faut constituer un jeu de données de **qualité homogène**, dont la structure et les formats sont cohérents et bien définis :

- 1  Traiter les valeurs manquantes
- 2  Supprimer les valeurs aberrantes
- 3  Redéfinir les échelles
- 4  Enrichir avec des référentiels

FIGURE 3.1 – Les actions principales du preprocessing

3.2 Description et visualisation des données

3.2.1 Pourquoi explorer avant de nettoyer ?

🎯 Objectif

Principe fondamental : On ne peut pas nettoyer efficacement ce qu'on ne comprend pas. L'exploration visuelle et statistique est **essentielle** avant toute opération de nettoyage.

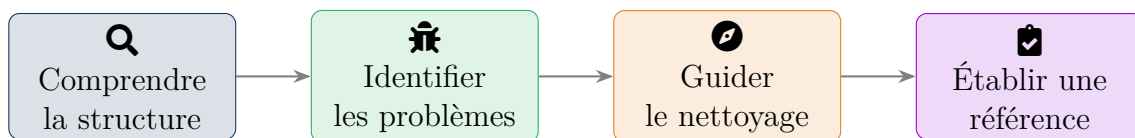


FIGURE 3.2 – Processus d'exploration préalable

3.2.2 Description statistique de base

Information	Python	Description
Forme du dataset	<code>df.shape</code>	(n_lignes, n_colonnes)
Types de données	<code>df.dtypes</code>	Type de chaque colonne
Stats descriptives	<code>df.describe()</code>	Moyenne, médiane, std...
Comptage catégories	<code>value_counts()</code>	Fréquence par catégorie

TABLE 3.1 – Commandes Python pour l'exploration

3.2.3 Code Python : Description statistique

```

1 import pandas as pd
2 import numpy as np
3
4 # Charger les donnees
5 df = pd.read_csv("data.csv")
6
7 # Forme du dataset
8 print("Shape:", df.shape)  # (lignes, colonnes)
9
10 # Types de donnees
11 print(df.dtypes)
12 df.info()
13
14 # Statistiques descriptives (numeriques)
15 print(df.describe())
16
17 # Pour variables categorielles
18 print(df['categorie'].value_counts())
19

```

```
20 # Verifications importantes
21 print("Valeurs manquantes:")
22 print(df.isnull().sum())
23
24 print("\nCardinalite:")
25 print(df.select_dtypes(include=['object']).nunique())
```

Listing 3.1 – Description statistique de base

3.3 Visualisation des données

3.3.1 Visualisation des valeurs manquantes

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3 import missingno as msno
4
5 # Heatmap des valeurs manquantes
6 msno.matrix(df)
7 plt.show()
8
9 # Barplot du pourcentage de manquants
10 missing_pct = (df.isnull().sum() / len(df)) * 100
11 missing_pct = missing_pct[missing_pct > 0].sort_values(ascending=
    False)
12
13 plt.figure(figsize=(10, 6))
14 missing_pct.plot(kind='bar')
15 plt.title("Pourcentage de valeurs manquantes par colonne")
16 plt.ylabel("Pourcentage (%)")
17 plt.show()
18
19 # Matrice de correlation des manquants
20 msno.heatmap(df)
21 plt.title("Correlation des valeurs manquantes")
22 plt.show()
```

Listing 3.2 – Visualisation des valeurs manquantes

3.3.2 Visualisation des distributions

Anatomie d'un Boxplot

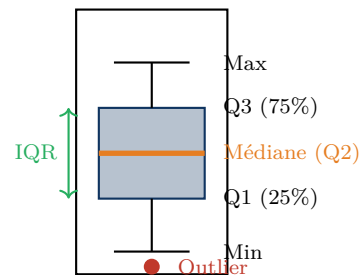


FIGURE 3.3 – Structure d'un boxplot pour la détection d'outliers

```

1 import matplotlib.pyplot as plt
2 import seaborn as sns
3 from scipy import stats
4
5 # Histogramme
6 df['age'].hist(bins=30, edgecolor='black')
7 plt.title("Distribution de l'age")
8 plt.xlabel("Age")
9 plt.ylabel("Frequence")
10 plt.show()
11
12 # Boxplot
13 plt.boxplot(df['age'].dropna())
14 plt.title("Boxplot de l'age")
15 plt.show()
16
17 # Q-Q plot pour tester la normalite
18 stats.probplot(df['age'].dropna(), dist="norm", plot=plt)
19 plt.title("Q-Q Plot")
20 plt.show()
21
22 # Densite (KDE)
23 df['age'].plot(kind='density')
24 plt.title("Densite de l'age")
25 plt.show()

```

Listing 3.3 – Visualisation des distributions

3.3.3 Visualisation des valeurs aberrantes (outliers)

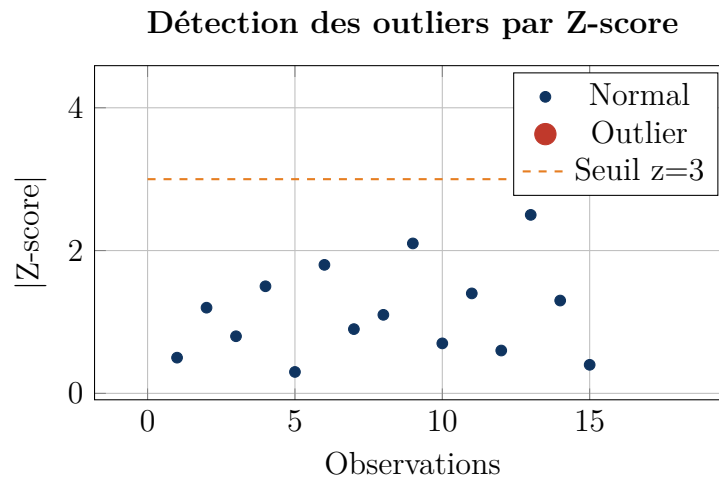


FIGURE 3.4 – Identification des outliers par Z-score

```

1 import numpy as np
2 from scipy import stats
3
4 # Boxplot pour detecter outliers
5 plt.boxplot(df['age'].dropna())
6 plt.title("Boxplot - Detection d'outliers")
7 plt.show()
8
9 # Scatter plot
10 plt.scatter(df['age'], df['salaire'])
11 plt.xlabel("Age")
12 plt.ylabel("Salaire")
13 plt.title("Scatter plot - Outliers visuels")
14 plt.show()
15
16 # Methode Z-score
17 z_scores = np.abs(stats.zscore(df['age'].dropna()))
18 outliers = df[z_scores > 3]
19 print(f"Nombre d'outliers (|z| > 3): {len(outliers)}")
20
21 # Visualisation Z-score
22 plt.scatter(range(len(df)), np.abs(stats.zscore(df['age'].fillna(0))))
23 plt.axhline(y=3, color='r', linestyle='--', label='Seuil z=3')
24 plt.title("Z-scores")
25 plt.ylabel("|Z-score|")
26 plt.legend()
27 plt.show()

```

Listing 3.4 – Détection des valeurs aberrantes

! Attention

Questions à se poser face à un outlier :

- Est-ce une erreur de saisie ou une valeur légitime ?
- Faut-il supprimer, transformer ou imputer ?
- Quel est son impact sur les statistiques ?

3.4 Les Données Aberrantes (Outliers) - Guide Complet

📖 Définition

Une **donnée aberrante** (ou *outlier*) est une observation qui s'écarte significativement des autres observations du jeu de données. Elle peut être le résultat d'une erreur de mesure, d'une erreur de saisie, ou représenter une variation naturelle extrême mais légitime.

3.4.1 Types de données aberrantes

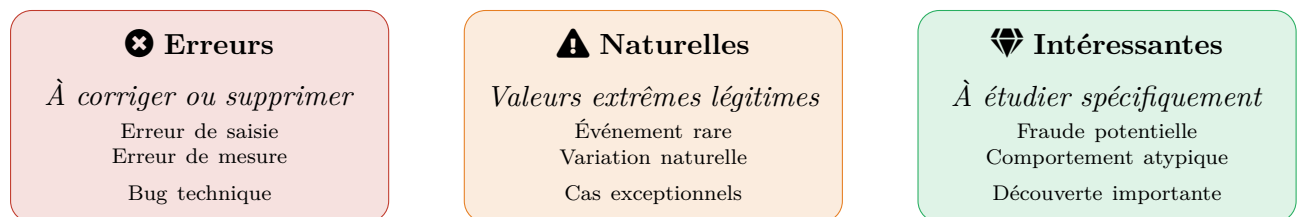


FIGURE 3.5 – Les trois types de données aberrantes

3.4.2 Exemples concrets de données aberrantes

💡 Exemple

Exemple 1 : Erreur de saisie

Dans un fichier de salaires d'employés :

Employé	Salaire (€)	Statut
Alice	2 500	Normal
Bob	3 200	Normal
Charlie	320 000	! Outlier !
David	2 800	Normal

→ Charlie gagne 320 000€ au lieu de 3 200€ → **Erreur de saisie** (un zéro en trop)

 Exemple**Exemple 2 : Valeur naturelle extrême**

Âges des participants à une conférence tech :

25	28	32	35	29	31	78	27
----	----	----	----	----	----	----	----

→ La personne de 78 ans est un **outlier légitime** : c'est un expert senior invité.
Ne pas supprimer car c'est une valeur réelle !

 Exemple**Exemple 3 : Détection de fraude**

Transactions bancaires d'un client :

Date	Montant (€)	Lieu
01/01	45	Paris
02/01	120	Paris
03/01	35	Paris
03/01	5 000	Nigeria
04/01	78	Paris

→ Transaction de 5 000€ au Nigeria → **Fraude potentielle** à investiguer !

3.4.3 Méthodes de détection des outliers

 Méthode 1 : Règle de l'IQR (Interquartile Range)

Principe : Une valeur est considérée comme aberrante si elle est en dehors de l'intervalle :

$$[Q1 - 1.5 \times IQR ; Q3 + 1.5 \times IQR]$$

Où : $IQR = Q3 - Q1$ (écart interquartile)**Exemple numérique :**

- Données : 10, 12, 14, 15, 16, 18, 20, 22, 25, **85**
- $Q1 = 13$, $Q3 = 21$, donc $IQR = 8$
- Bornes : $[13 - 12 ; 21 + 12] = [1 ; 33]$
- **85** est hors de $[1, 33]$ → **Outlier détecté !**

```

1 import pandas as pd
2 import numpy as np
3
4 # Donnees exemple
5 salaires = [2500, 3200, 2800, 3100, 320000, 2900, 3000]
6 df = pd.DataFrame({'salaire': salaires})
7
8 # Calcul IQR
9 Q1 = df['salaire'].quantile(0.25)
10 Q3 = df['salaire'].quantile(0.75)
```

```

11 IQR = Q3 - Q1
12
13 # Bornes
14 borne_inf = Q1 - 1.5 * IQR
15 borne_sup = Q3 + 1.5 * IQR
16
17 print(f"Q1 = {Q1}, Q3 = {Q3}, IQR = {IQR}")
18 print(f"Intervalle acceptable: [{borne_inf}, {borne_sup}]")
19
20 # Detection des outliers
21 outliers = df[(df['salaire'] < borne_inf) | (df['salaire'] >
    borne_sup)]
22 print(f"\nOutliers detectes:")
23 print(outliers)
24 # Resultat: 320000 est un outlier!

```

Listing 3.5 – Détection par IQR avec Python

Méthode 2 : Z-Score

Principe : Le Z-score mesure le nombre d'écarts-types entre une valeur et la moyenne.

$$Z = \frac{x - \mu}{\sigma}$$

Règle : Si $|Z| > 3$, la valeur est considérée comme aberrante.

Exemple :

- Moyenne $\mu = 3000$, Écart-type $\sigma = 300$
- Pour $x = 320000$: $Z = \frac{320000 - 3000}{300} = 1056.7$
- $|1056.7| \gg 3 \rightarrow$ **Outlier évident !**

```

1 from scipy import stats
2 import numpy as np
3
4 # Donnees
5 salaires = np.array([2500, 3200, 2800, 3100, 320000, 2900, 3000])
6
7 # Calcul des Z-scores
8 z_scores = stats.zscore(salaires)
9
10 print("Salaire\t\tZ-score\t\tOutlier?")
11 print("-" * 45)
12 for salaire, z in zip(salaires, z_scores):
13     is_outlier = "OUI" if abs(z) > 3 else "Non"
14     print(f"{salaire}\t\t{z:.2f}\t\t{is_outlier}")
15
16 # Filtrer les outliers
17 mask = np.abs(z_scores) > 3
18 outliers = salaires[mask]
19 print(f"\nOutliers (|Z| > 3): {outliers}")

```

Listing 3.6 – Détection par Z-score avec Python

3.4.4 Traitement des outliers

Stratégie	Quand l'utiliser	Exemple Python
Suppression	Erreurs évidentes, peu de données concernées	<code>df = df[z_scores < 3]</code>
Winsorisation	Garder les données mais limiter leur impact	<code>df['col'].clip(lower=Q1, upper=Q3)</code>
Transformation log	Distributions très asymétriques	<code>df['col'] = np.log1p(df['col'])</code>
Imputation	Remplacer par médiane/moyenne	<code>df.loc[mask, 'col'] = median</code>
Séparation	Analyse spécifique des outliers	<code>df_outliers = df[mask]</code>

TABLE 3.2 – Stratégies de traitement des outliers

```

1 import pandas as pd
2 import numpy as np
3
4 # Donnees avec outliers
5 df = pd.DataFrame({
6     'salaire': [2500, 3200, 2800, 320000, 2900, 3000, -500]
7 })
8
9 # 1. SUPPRESSION des outliers
10 Q1, Q3 = df['salaire'].quantile([0.25, 0.75])
11 IQR = Q3 - Q1
12 mask = (df['salaire'] >= Q1 - 1.5*IQR) & (df['salaire'] <= Q3 +
13     1.5*IQR)
14 df_clean = df[mask].copy()
15 print("Apres suppression:", df_clean['salaire'].tolist())
16
17 # 2. WINSORISATION (capper les extremes)
18 df['salaire_winsor'] = df['salaire'].clip(
19     lower=df['salaire'].quantile(0.05),
20     upper=df['salaire'].quantile(0.95)
21 )
22
23 # 3. TRANSFORMATION LOG (pour asymetrie positive)
24 df['salaire_log'] = np.log1p(df['salaire'].clip(lower=0))

```

```

24
25 # 4. REMPLACEMENT par la mediane
26 mediane = df['salaire'].median()
27 df['salaire_impute'] = df['salaire'].copy()
28 df.loc[~mask, 'salaire_impute'] = mediane
29
30 print(df)

```

Listing 3.7 – Traitement complet des outliers

3.5 L'Encodage des Variables Catégorielles - Guide Complet

Définition

L'**encodage** est le processus de transformation des variables catégorielles (texte) en valeurs numériques. C'est une étape obligatoire car les algorithmes de Machine Learning ne peuvent traiter que des nombres.

Important

Pourquoi encoder ?

- Les algorithmes ML ne comprennent que les **nombres**
- Une variable comme "Rouge", "Vert", "Bleu" doit devenir 0, 1, 2 ou être transformée en colonnes binaires
- Un mauvais encodage peut créer des **relations artificielles** entre les catégories

3.5.1 Types d'encodage

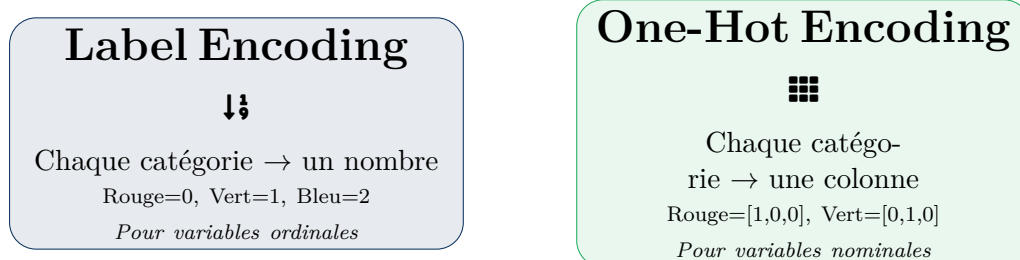


FIGURE 3.6 – Les deux principales méthodes d'encodage

3.5.2 Label Encoding - Encodage par étiquettes

Définition

Le **Label Encoding** attribue un entier unique à chaque catégorie. Simple mais crée un ordre artificiel entre les catégories.

 Exemple**Exemple : Niveau d'études (variable ordinale)**

Données originales	→	Après Label Encoding
Bac	→	0
Licence	→	1
Master	→	2
Doctorat	→	3

✔ **Approprié** car il y a un ordre naturel : $Bac < Licence < Master < Doctorat$

 Attention**Danger du Label Encoding avec des variables nominales !**

Si on encode les couleurs : Rouge=0, Vert=1, Bleu=2

L'algorithme peut interpréter : $Bleu > Vert > Rouge$ ou $Rouge + Vert = Bleu$

Ce qui n'a **aucun sens** ! → Utiliser One-Hot Encoding à la place.

```

1 from sklearn.preprocessing import LabelEncoder
2 import pandas as pd
3
4 # Donnees
5 df = pd.DataFrame({
6     'niveau_etudes': ['Bac', 'Licence', 'Master', 'Bac', '
7     Doctorat', 'Master']
8 })
9
10 # Methode 1: LabelEncoder de sklearn
11 le = LabelEncoder()
12 df['niveau_encoded'] = le.fit_transform(df['niveau_etudes'])
13
14 print("Avant et apres encodage:")
15 print(df)
16
17 # Voir la correspondance
18 print("\nCorrespondance:")
19 for i, label in enumerate(le.classes_):
20     print(f" {label} -> {i}")
21
22 # Methode 2: Mapping manuel (recommande pour ordinales)
23 mapping = {'Bac': 0, 'Licence': 1, 'Master': 2, 'Doctorat': 3}
24 df['niveau_manuel'] = df['niveau_etudes'].map(mapping)
25 print("\nAvec mapping manuel:")
26 print(df)

```

Listing 3.8 – Label Encoding avec Python

3.5.3 One-Hot Encoding - Encodage binaire

Définition

Le **One-Hot Encoding** crée une nouvelle colonne binaire (0 ou 1) pour chaque catégorie. Élimine tout ordre artificiel entre les catégories.

Exemple

Exemple : Couleur de voiture (variable nominale)

Données originales :

ID	Couleur
1	Rouge
2	Bleu
3	Vert
4	Rouge

Après One-Hot Encoding :

ID	Couleur_Rouge	Couleur_Bleu	Couleur_Vert
1	1	0	0
2	0	1	0
3	0	0	1
4	1	0	0

 Chaque ligne a exactement un "1" dans les colonnes de couleur.

```
1 import pandas as pd
2
3 # Donnees
4 df = pd.DataFrame({
5     'id': [1, 2, 3, 4],
6     'couleur': ['Rouge', 'Bleu', 'Vert', 'Rouge'],
7     'prix': [15000, 18000, 16000, 14500]
8 })
9
10 print("Donnees originales:")
11 print(df)
12
13 # Methode 1: pd.get_dummies (simple et efficace)
14 df_encoded = pd.get_dummies(df, columns=['couleur'], prefix='
    couleur')
15 print("\nAprès One-Hot Encoding:")
16 print(df_encoded)
17
18 # Methode 2: OneHotEncoder de sklearn
19 from sklearn.preprocessing import OneHotEncoder
20
21 encoder = OneHotEncoder(sparse_output=False, handle_unknown='
    ignore')
```

```

22 couleurs_encoded = encoder.fit_transform(df[['couleur']])
23
24 # Créer un DataFrame avec les nouvelles colonnes
25 colonnes = encoder.get_feature_names_out(['couleur'])
26 df_ohe = pd.DataFrame(couleurs_encoded, columns=colonnes)
27
28 # Concatener avec les autres colonnes
29 df_final = pd.concat([df[['id', 'prix']], df_ohe], axis=1)
30 print("\nAvec sklearn OneHotEncoder:")
31 print(df_final)

```

Listing 3.9 – One-Hot Encoding avec Python

3.5.4 Comparaison et choix de la méthode

Critère	Label Encoding	One-Hot Encoding
Type de variable	Variables ordinales (avec ordre)	Variables nominales (sans ordre)
Nombre de colonnes	1 colonne	n colonnes (= nb catégories)
Avantage	Simple, peu de mémoire	Pas d'ordre artificiel
Inconvénient	Crée un ordre artificiel	Explosion dimensionnelle
Exemple adapté	Taille (S<M<L<XL)	Pays, Couleur, Marque

TABLE 3.3 – Comparaison Label Encoding vs One-Hot Encoding

Astuce

Règle simple pour choisir :

- ➔ Variable avec **ordre naturel** (petit < moyen < grand) ➔ **Label Encoding**
- ➔ Variable **sans ordre** (rouge, vert, bleu) ➔ **One-Hot Encoding**
- ➔ Beaucoup de catégories (>50) ➔ Considérer **Target Encoding**

3.5.5 Autres méthodes d'encodage avancées

```

1 import pandas as pd
2 import numpy as np
3
4 df = pd.DataFrame({
5     'ville': ['Paris', 'Lyon', 'Paris', 'Marseille', 'Lyon', 'Paris'],
6     'ventes': [100, 80, 120, 60, 90, 110]

```

```

7 })
8
9 # 1. FREQUENCY ENCODING - Encodage par frequence
10 freq = df['ville'].value_counts(normalize=True)
11 df['ville_freq'] = df['ville'].map(freq)
12 print("Frequency Encoding:")
13 print(df[['ville', 'ville_freq']])
14
15 # 2. TARGET ENCODING - Encodage par la moyenne de la cible
16 target_mean = df.groupby('ville')['ventes'].mean()
17 df['ville_target'] = df['ville'].map(target_mean)
18 print("\nTarget Encoding:")
19 print(df[['ville', 'ventes', 'ville_target']])
20
21 # 3. BINARY ENCODING - Pour beaucoup de categories
22 # Encode en binaire puis separe les bits
23 from sklearn.preprocessing import LabelEncoder
24 le = LabelEncoder()
25 df['ville_label'] = le.fit_transform(df['ville'])
26 # Conversion en binaire
27 max_bits = int(np.ceil(np.log2(df['ville_label'].max() + 1)))
28 for i in range(max_bits):
29     df[f'ville_bit_{i}'] = (df['ville_label'] >> i) & 1
30 print("\nBinary Encoding:")
31 print(df[['ville', 'ville_label', 'ville_bit_0', 'ville_bit_1']])

```

Listing 3.10 – Méthodes d'encodage avancées

⚠ Important**Attention au data leakage avec Target Encoding !**

Le Target Encoding utilise la variable cible pour encoder. Il faut :

- Calculer les moyennes **uniquement** sur le train set
- Appliquer ces moyennes au test set
- Utiliser des techniques de régularisation (smoothing)

3.5.6 Exemple complet : Pipeline d'encodage

```

1 import pandas as pd
2 from sklearn.preprocessing import LabelEncoder, OneHotEncoder
3 from sklearn.compose import ColumnTransformer
4 from sklearn.pipeline import Pipeline
5
6 # Donnees exemple
7 df = pd.DataFrame({
8     'taille': ['S', 'M', 'L', 'XL', 'M', 'S'],          # Ordinale
9     'couleur': ['Rouge', 'Bleu', 'Rouge', 'Vert', 'Bleu', 'Rouge'],
10     'marque': ['Nike', 'Adidas', 'Nike', 'Puma', 'Nike', 'Adidas'],
11     # Nominale

```

```

11     'prix': [50, 60, 55, 70, 65, 45]
12 })
13
14 print("Donnees originales:")
15 print(df)
16
17 # 1. Label Encoding pour 'taille' (ordinaire)
18 taille_mapping = {'S': 0, 'M': 1, 'L': 2, 'XL': 3}
19 df['taille_encoded'] = df['taille'].map(taille_mapping)
20
21 # 2. One-Hot Encoding pour 'couleur' et 'marque' (nominales)
22 df_final = pd.get_dummies(df, columns=['couleur', 'marque'],
23                               drop_first=True) # drop_first evite la
24                               multicollinearite
25
26 print("\nApres encodage complet:")
27 print(df_final)
28
29 # Verification des types
30 print("\nTypes des colonnes:")
31 print(df_final.dtypes)

```

Listing 3.11 – Pipeline complet d'encodage

3.5.7 Visualisation récapitulative de l'encodage

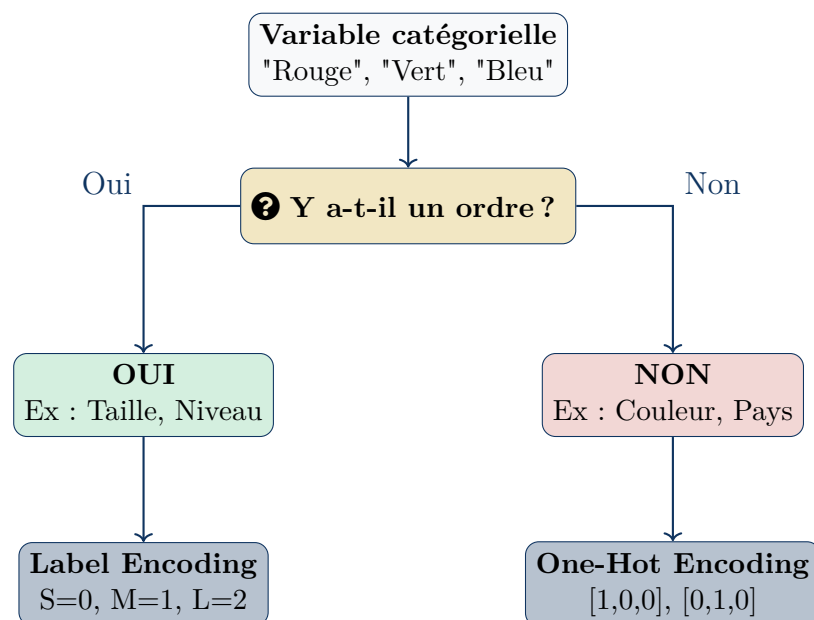


FIGURE 3.7 – Arbre de décision pour choisir la méthode d'encodage

3.5.8 Visualisation des corrélations

```

1 import seaborn as sns
2

```

```

3 # Calcul de la matrice de corrélation
4 corr_matrix = df.select_dtypes(include=[np.number]).corr()
5
6 # Heatmap
7 plt.figure(figsize=(10, 8))
8 sns.heatmap(corr_matrix, annot=True, cmap='coolwarm',
9             center=0, fmt='.2f', linewidths=0.5)
10 plt.title("Matrice de corrélation")
11 plt.show()
12
13 # Pair plots (scatter plots multiples)
14 numeric_cols = df.select_dtypes(include=[np.number]).columns[:5]
15 sns.pairplot(df[numeric_cols])
16 plt.show()

```

Listing 3.12 – Visualisation des corrélations

3.5.9 Visualisation des incohérences

```

1 from datetime import datetime
2
3 # Formats inconsistants - casse
4 print("Valeurs uniques (casse):")
5 print(df['ville'].unique())
6
7 # Valeurs hors domaine
8 invalid_age = df[(df['age'] < 0) | (df['age'] > 120)]
9 print(f"Ages invalides: {len(invalid_age)}")
10
11 # Relations logiques - dates incoherentes
12 df['date_achat'] = pd.to_datetime(df['date_achat'])
13 df['date_inscription'] = pd.to_datetime(df['date_inscription'])
14 incoherent = df[df['date_achat'] < df['date_inscription']]
15 print(f"Dates incoherentes: {len(incoherent)}")
16
17 # Graphique temporel
18 plt.plot(df['date_achat'], df['montant'], 'o')
19 plt.xlabel("Date d'achat")
20 plt.ylabel("Montant")
21 plt.title("Verification des incoherences temporelles")
22 plt.xticks(rotation=45)
23 plt.show()
24
25 # Tableau croise
26 print(pd.crosstab(df['categorie'], df['statut'], margins=True))

```

Listing 3.13 – Détection des incohérences

3.6 Checklist d'exploration

Checklist d'exploration avant nettoyage

- ✓ **Structure** : Forme, types, mémoire utilisée
- ✓ **Valeurs manquantes** : Pourcentage, pattern (MCAR/MAR/M-NAR)
- ✓ **Distributions** : Forme, symétrie, normalité
- ✓ **Valeurs aberrantes** : Identification et quantification
- ✓ **Cardinalité** : Nombre de modalités pour variables catégorielles
- ✓ **Corrélations** : Relations entre variables
- ✓ **Incohérences** : Formats, valeurs hors domaine, logique métier
- ✓ **Doublons** : Fréquence et nature

Astuce

Documentation : Toujours documenter les problèmes identifiés et les décisions prises pour le nettoyage. Cela facilite la reproductibilité et la maintenance du projet.

3.7 Outils et bibliothèques recommandées

Outil	Usage	Fonctions clés
Pandas	Manipulation	<code>describe()</code> , <code>info()</code> , <code>isnull()</code>
NumPy	Calculs	<code>array</code> , <code>mean()</code> , <code>std()</code>
Matplotlib	Visualisation	<code>hist()</code> , <code>boxplot()</code> , <code>scatter()</code>
Seaborn	Visualisation+	<code>heatmap()</code> , <code>pairplot()</code>
Missingno	Manquants	<code>matrix()</code> , <code>heatmap()</code> , <code>bar()</code>
Sweetviz	Rapports auto	Génération de rapports complets

TABLE 3.4 – Principales bibliothèques Python pour le preprocessing

Chapitre 4

Résumé et Points Clés

4.1 Ce qu'il faut retenir



FIGURE 4.1 – Les trois étapes fondamentales

4.2 Points clés à retenir

Points clés du cours

1. **Le preprocessing est crucial** : 60-80% du temps d'un projet ML
2. **Garbage in, garbage out** : La qualité des données détermine la qualité du modèle
3. **Connaître ses données** : Information → Donnée → Indicateur
4. **Types de variables** : Qualitatives (nominales, ordinales) vs Quantitatives (discrètes, continues)
5. **Explorer avant d'agir** : Toujours visualiser et décrire avant de nettoyer
6. **Les 5 piliers** : Manquants, Nettoyage, Transformation, Encodage, Enrichissement
7. **Documenter** : Garder une trace des décisions prises

Merci pour votre attention !

Tacko CISS

Étudiante en Master 1 Systèmes d'Information